

Cross-Platform Mobile App Development

Week 6 — React Native (Part 2): Navigation, State Management & Animations

Instructor: Nguyen Thanh Tuan, PhD

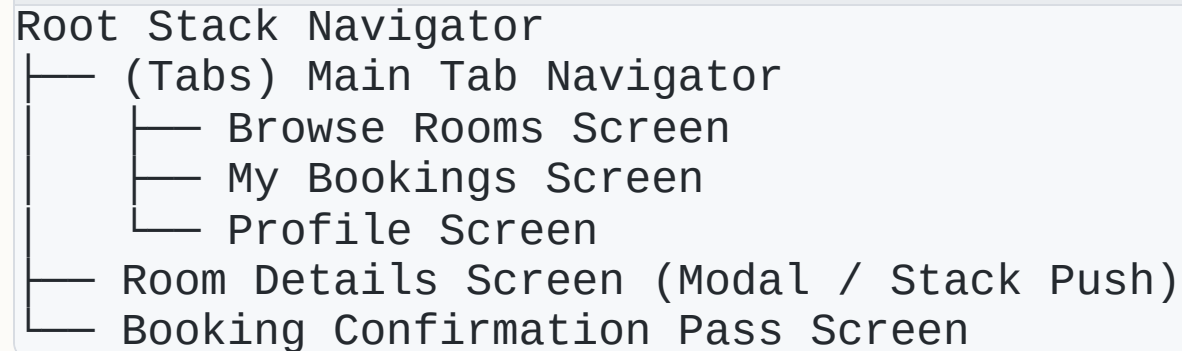
Institution: Faculty of Computer Science, VKU

Today's Agenda

1. React Navigation Architecture: Stack, Bottom Tabs, and Drawer Navigators
2. Type-Safe Navigation with TypeScript
3. Global State Management: Why Zustand beats Redux for Mobile
4. Server State & Remote Caching with TanStack Query
5. Declarative Mobile Animations with React Native Reanimated
6. Native Gesture Handling (`react-native-gesture-handler`)
7. Mini-Project 2 Submission Review

Part 1: Navigation Architecture

React Navigation Structure



```
graph TD; Root[Root Stack Navigator] --- Tabs["(Tabs) Main Tab Navigator"]; Root --- RoomDetails["Room Details Screen (Modal / Stack Push)"]; Root --- Booking["Booking Confirmation Pass Screen"]; Tabs --- Browse[Browse Rooms Screen]; Tabs --- Bookings["My Bookings Screen"]; Tabs --- Profile["Profile Screen"];
```

Root Stack Navigator

- (Tabs) Main Tab Navigator
 - Browse Rooms Screen
 - My Bookings Screen
 - Profile Screen
- Room Details Screen (Modal / Stack Push)
- Booking Confirmation Pass Screen

Why this pattern?

- **Stack** handles screen-to-screen transitions (push/pop with back gesture).
- **Bottom Tabs** provide persistent top-level navigation.
- **Nesting Tabs inside Stack** allows modal screens to cover the tab bar.

Setup & Dependencies

```
# Core navigation
npm install @react-navigation/native

# Required peer dependencies for Expo
npx expo install react-native-screens react-native-safe-area-context

# Navigator types
npm install @react-navigation/native-stack
npm install @react-navigation/bottom-tabs
```

```
// App.tsx – Wrap everything in NavigationContainer
import { NavigationContainer } from '@react-navigation/native';

export default function App() {
  return (
    <NavigationContainer>
      <RootStack />
    </NavigationContainer>
  );
}
```

Type-Safe Route Parameters

```
import { NativeStackScreenProps } from '@react-navigation/native-stack';

// 1. Define all routes and their param shapes
type RootStackParamList = {
  MainTabs: undefined;
  RoomDetails: { roomId: string; roomName: string };
  BookingConfirmation: { bookingId: string };
};

// 2. Generate typed props for each screen
type RoomDetailsProps =
  NativeStackScreenProps<RootStackParamList, 'RoomDetails'>;

// 3. Use typed props in the screen component
function RoomDetailsScreen({ route, navigation }: RoomDetailsProps) {
  const { roomId, roomName } = route.params; // ✅ Fully typed!
  // ...
}
```



Without `RootStackParamList`, typos in route names become **silent runtime crashes**.

Stack Navigator Implementation

```
import { createNativeStackNavigator } from '@react-navigation/native-stack';

const Stack = createNativeStackNavigator<RootStackParamList>();

function RootStack() {
  return (
    <Stack.Navigator initialRouteName="MainTabs"
      screenOptions={{
        headerStyle: { backgroundColor: '#1E3A5F' },
        headerTintColor: '#fff',
        animation: 'slide_from_right',
      }}>
      <Stack.Screen name="MainTabs" component={MainTabNavigator}
        options={{ headerShown: false }} />
      <Stack.Screen name="RoomDetails" component={RoomDetailsScreen}
        options={({ route }) => ({ title: route.params.roomName })} />
      <Stack.Screen name="BookingConfirmation"
        component={BookingConfirmationScreen}
        options={{ presentation: 'modal' }} />
    </Stack.Navigator>
  );
}
```

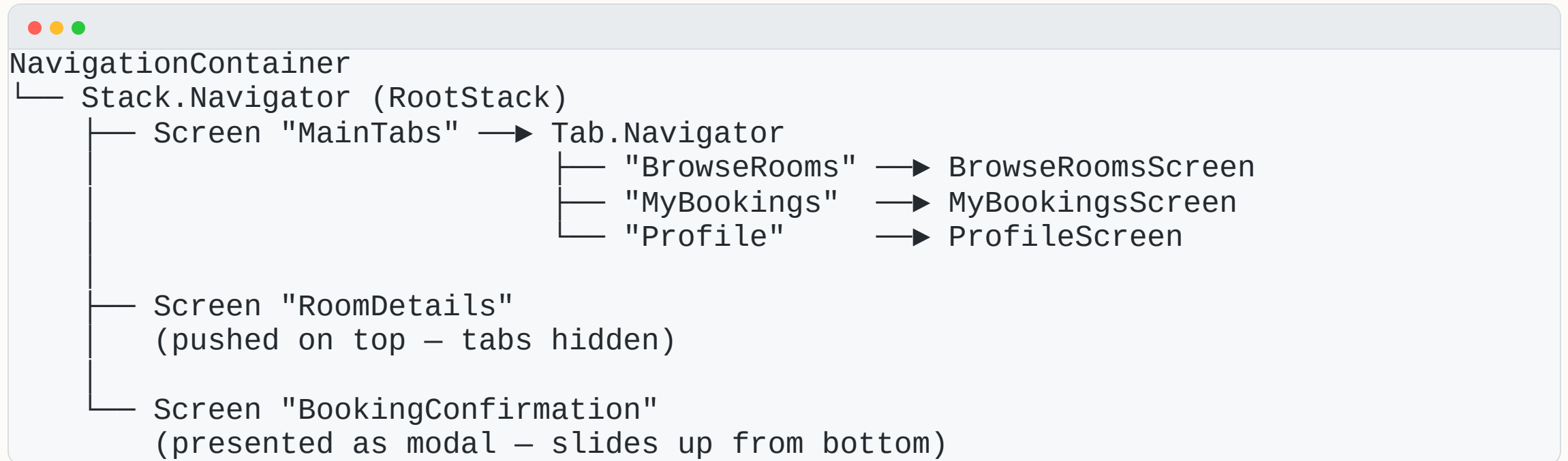
Bottom Tab Navigator

```
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs',
import { Ionicons } from '@expo/vector-icons';

const Tab = createBottomTabNavigator<TabParamList>();

function MainTabNavigator() {
  return (
    <Tab.Navigator screenOptions={({ route }) => ({
      tabBarIcon: ({ focused, color, size }) => {
        const icons = {
          BrowseRooms: focused ? 'search' : 'search-outline',
          MyBookings:  focused ? 'calendar' : 'calendar-outline',
          Profile:      focused ? 'person' : 'person-outline',
        };
        return <Ionicons name={icons[route.name]} size={size}
                          color={color} />;
      },
      tabBarActiveTintColor: '#3B82F6',
    })}>
      <Tab.Screen name="BrowseRooms" component={BrowseRoomsScreen} />
      <Tab.Screen name="MyBookings" component={MyBookingsScreen} />
      <Tab.Screen name="Profile" component={ProfileScreen} />
    </Tab.Navigator>
  );
}
```

Nesting Navigators: Full Architecture



Key pattern: Screens **outside** the Tab Navigator push on top and **hide the tab bar**. Screens **inside** keep tabs visible.

Part 2: State Management with Zustand

Why Zustand > Redux for Mobile?

Aspect	Redux Toolkit	Zustand
Boilerplate	Slice + Reducer + Action + Provider	Single <code>create()</code> call
Bundle size	~12 KB (+ React-Redux)	~1.5 KB
Provider	✓ Must wrap <code><Provider></code>	✗ No provider needed
Re-render	<code>useSelector</code> + memoize	Built-in selectors
Learning curve	Steep	Minimal

For mobile apps with < 15 screens, Zustand is almost always the better choice.

Zustand Store: Definition

```
import { create } from 'zustand',
import { persist, createJSONStorage } from 'zustand/middleware';
import AsyncStorage from '@react-native-async-storage/async-storage';

interface BookingState {
  bookings: Booking[];
  addBooking: (booking: Booking) => void;
  cancelBooking: (id: string) => void;
}

export const useBookingStore = create<BookingState>()(
  persist(
    (set) => ({
      bookings: [],
      addBooking: (b) =>
        set((s) => ({ bookings: [...s.bookings, b] })),
      cancelBooking: (id) =>
        set((s) => ({
          bookings: s.bookings.filter((b) => b.id !== id),
        })),
    }),
    {
      name: 'vku-booking-storage',
      storage: createJSONStorage(() => AsyncStorage)
    }
  )
);
```

Zustand Store: Consuming with Selectors

```
function MyBookingsScreen() {  
  //  GOOD: Select only what you need  
  const bookings = useBookingStore((s) => s.bookings);  
  const cancelBooking = useBookingStore((s) => s.cancelBooking);  
  
  //  BAD: Subscribes to entire store → re-renders on ANY change  
  // const store = useBookingStore();  
  
  const active = bookings.filter((b) => b.status === 'confirmed');  
  
  return (  
    <FlatList  
      data={active}  
      keyExtractor={(item) => item.id}  
      renderItem={({ item }) => (  
        <BookingCard booking={item}  
          onCancel={() => cancelBooking(item.id)} />  
      )}  
      ListEmptyComponent={<EmptyState message="No bookings!" />}  
    />  
  );  
}
```

Part 3: Server State with TanStack Query

Client State vs. Server State

	Zustand (Client State)	TanStack Query (Server State)
What	UI state, preferences	API data, database records
Lifecycle	Lives while app is open	Stale → refetch → cache
Examples	Theme, filters, auth	Room list, booking history
Caching	Manual	Automatic with <code>staleTime</code>
Loading/Error	Manual booleans	Built-in <code>isLoading</code> , <code>isError</code>

If the data exists on a server and needs syncing, use TanStack Query. If it's local-only, use Zustand.

TanStack Query: Setup

```
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';

const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 5 * 60 * 1000, // Fresh for 5 minutes
      gcTime: 10 * 60 * 1000, // GC after 10 minutes
      retry: 2,
    },
  },
});

export function QueryProvider({ children }: { children: React.ReactNode }) {
  return (
    <QueryClientProvider client={queryClient}>
      {children}
    </QueryClientProvider>
  );
}
```

TanStack Query: Custom Hook

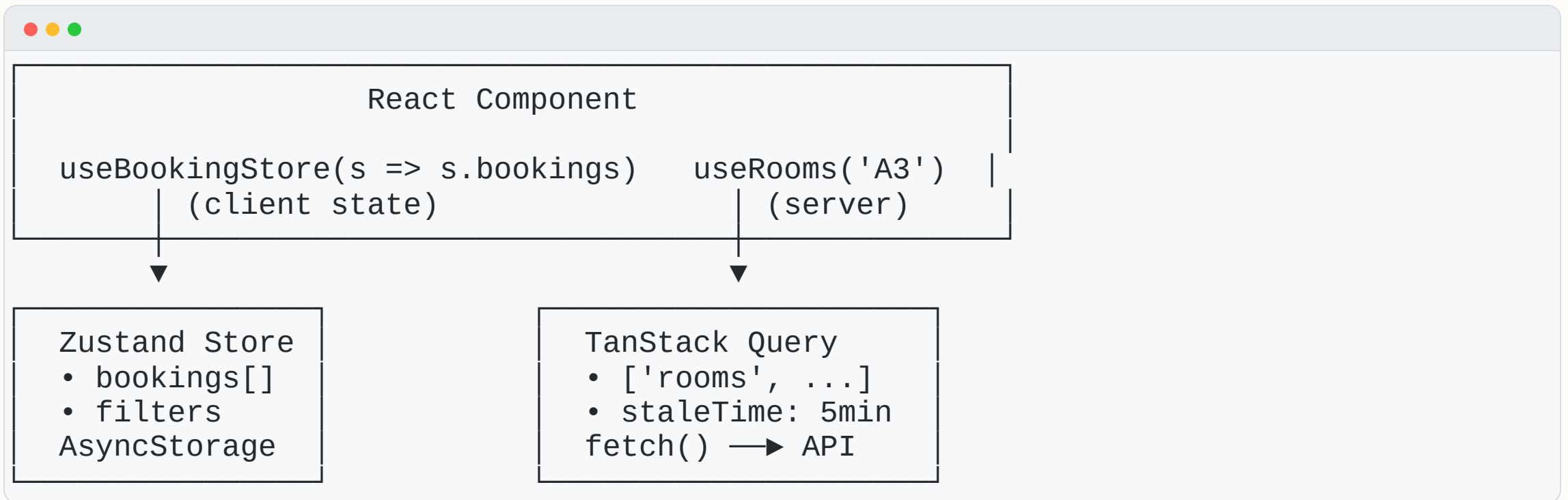
```
import { useQuery } from '@tanstack/react-query';

export function useRooms(building?: string) {
  return useQuery({
    queryKey: ['rooms', { building }], // Cache key
    queryFn: async () => {
      const res = await fetch(
        `https://api.vku.edu.vn/rooms?building=${building ?? ''}`
      );
      if (!res.ok) throw new Error('Failed to fetch');
      return res.json() as Promise<Room[]>;
    },
  });
}
```

TanStack Query: Usage in Screen

```
function BrowseRoomsScreen() {  
  const [building, setBuilding] = useState<string>();  
  const { data: rooms, isLoading, isError, refetch } = useRooms(building);  
  
  if (isLoading) return <LoadingSpinner />;  
  if (isError) return <ErrorBanner onRetry={refetch} />;  
  
  return (  
    <FlatList  
      data={rooms}  
      keyExtractor={(item) => item.id}  
      renderItem={({ item }) => <RoomCard room={item} />}  
      refreshing={isLoading}  
      onRefresh={refetch} // Pull-to-refresh!  
    />  
  );  
}
```

Data Flow: Zustand + TanStack Query



Part 4: Animations with Reanimated

React Native Reanimated 3

Why not **Animated** from core React Native?

- Core **Animated** runs on JS thread → frame drops during computation.
- **Reanimated** runs on the **UI thread** via worklets → guaranteed **60/120fps**.

```
npn expo install react-native-reanimated
```

Primitive	Purpose
<code>useSharedValue(init)</code>	Mutable value on UI thread (no re-render)
<code>useAnimatedStyle(() => {...})</code>	Style derived from shared values
<code>withTiming</code> / <code>withSpring</code>	Animation drivers

Layout Animations: Staggered Card Entry

```
import Animated, { FadeInDown, FadeOutUp, Layout }
  from 'react-native-reanimated';

function RoomCard({ room, index }: { room: Room; index: number }) {
  return (
    <Animated.View
      entering={FadeInDown.delay(index * 80).springify()}
      exiting={FadeOutUp.duration(200)}
      layout={Layout.springify()}
      style={styles.card}
    >
      <Text style={styles.roomName}>{room.name}</Text>
      <Text style={styles.location}>! {room.building}</Text>
    </Animated.View>
  );
}
```

💡 **Layout Animations** are the easiest path to premium-feeling list animations — just add `entering`, `exiting`, and `layout` props.

Custom Animated Interactions

```
import Animated, {
  useSharedValue, useAnimatedStyle, withSpring
} from 'react-native-reanimated';

function BookButton({ onPress }: { onPress: () => void }) {
  const scale = useSharedValue(1);

  const animStyle = useAnimatedStyle(() => ({
    transform: [{ scale: scale.value }],
  }));

  return (
    <Pressable
      onPressIn={() => { scale.value = withSpring(0.95); }}
      onPressOut={() => { scale.value = withSpring(1); }}
      onPress={onPress}>
      <Animated.View style={[styles.bookBtn, animStyle]}>
        <Text style={styles.bookBtnText}>Book This Room</Text>
      </Animated.View>
    </Pressable>
  );
}
```

Part 5: Gesture Handling

React Native Gesture Handler

```
npx expo install react-native-gesture-handler
```

Why not **PanResponder** ?

- **PanResponder** runs on JS thread → laggy, can't compete with scroll.
- **Gesture Handler** runs on **native thread** → buttery smooth.

Swipe-to-Cancel: Gesture + Reanimated

```
import { useSharedValue, useAnimatedStyle,
  withSpring, runOnJS } from 'react-native-reanimated';

function SwipeToCancel({ bookingId, onCancel }) {
  const translateX = useSharedValue(0);

  const pan = Gesture.Pan()
    .activeOffsetX(-10)
    .onUpdate((e) => {
      translateX.value = Math.min(0, e.translationX);
    })
    .onEnd((e) => {
      if (e.translationX < -120) runOnJS(onCancel)(bookingId);
      translateX.value = withSpring(0);
    });

  const style = useAnimatedStyle(() => ({
    transform: [{ translateX: translateX.value }],
  }));

  return (
    <GestureDetector gesture={pan}>
      <Animated.View style={[styles.card, style]}>
        /* Card content */
      </Animated.View>
    </GestureDetector>
  );
}
```

Part 6: Mini-Project 2 Submission

Mini-Project 2 Deliverables

1.  **Live Demo & Video:** Expo Go QR code link + 2–3 min video demo on a physical phone.
2.  **GitHub Repository:** Public React Native (Expo) repo with `README.md` setup instructions.
3.  **Short Technical Report (PDF):** 2–4 pages adhering to the official template.
4. **Deadline:** End of Week 6 (Sunday 23:59).

Grading Rubric

Criterion	Weight	Excellent (9-10)	Needs Work (< 7)
UI/UX	25%	Polished, animations	Unstyled / broken
Features	30%	Search + filter + booking	Missing key features
Navigation	15%	Stack + Tabs, typed params	Single screen
State	15%	Zustand + TanStack Query	No state management
Code Quality	15%	TypeScript strict, hooks	No types, monolithic

Today's Takeaways



React Navigation with Stack + Bottom Tabs covers 90% of mobile nav patterns.



Always define **ParamList** types — untyped navigation is a crash waiting to happen.



Zustand for client state + **TanStack Query** for server state — don't mix them.



Reanimated 3 Layout Animations = easiest path to premium-feeling UI.



Gesture Handler runs on native thread — always prefer it over **PanResponder**.

Questions & Homework

1. Implement Stack + Bottom Tabs navigation for the Room Booking app.
2. Create a Zustand store with `persist` middleware for bookings.
3. Add pull-to-refresh `FlatList` powered by TanStack Query.
4. Bonus: Add `FadeInDown` entering animations to room cards.

Q&A

Next Week: Flutter & Dart

- In Week 7, we begin **Flutter and Dart**!
- New language, new paradigm (Widget Tree), new engine (Impeller).
- Bring your curiosity — and uninstall your React habits temporarily 😊